

BRAC's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 6

Roll no: 32 PRN No: 12414914 Name of Student: **Rashi Gundawar**

Subject Name : Deep Learning

Problem Statement : Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm :

Step 1: Import the required libraries such as TensorFlow/Keras, NumPy, Pandas, Matplotlib, and Scikit-learn for image processing, CNN model development, training, evaluation, and visualization.

Step 2: Download and load the Tomato PlantVillage disease dataset containing images of healthy and diseased tomato leaves.

Step 3: Locate the Tomato dataset directory and identify the different disease classes available in the dataset.

Step 4: Preprocess the images by resizing them to 128×128 pixels, creating training and validation datasets, and dividing the data using an appropriate validation split.

Step 5: Select a smaller portion of the dataset to reduce computational time and display sample tomato leaf images with their corresponding disease labels.

Step 6: Design a CNN model consisting of multiple convolutional layers, ReLU activation functions, max-pooling layers, a flattening layer, fully connected layers, and a dropout layer.

Step 7: Add a Softmax output layer with neurons corresponding to the number of tomato disease classes.

Step 8: Compile the CNN model using the Adam optimizer, Sparse Categorical Cross-Entropy loss function, and accuracy as the evaluation metric.

Step 9: Train the CNN model using the training dataset for 5 epochs and validate its performance using the validation dataset.

Step 10: Plot the training and validation accuracy and loss curves to analyze the learning performance and identify possible overfitting.

Step 11: Generate predictions for the validation images using the trained CNN model.

Step 12: Evaluate the model using Accuracy, Precision, Recall, and F1-score.

Step 13: Generate a classification report and confusion matrix to analyze the classification performance for each tomato disease class.

Step 14: Display a sample tomato leaf image along with its actual disease class and the class predicted by the CNN model.

Step 15: Analyze the obtained results and determine the effectiveness of the CNN model in classifying tomato plant diseases.

Code :

```
# 1. Get class names from dataset folders
import os

class_names = sorted([
    folder for folder in os.listdir(tomato_path)
    if os.path.isdir(os.path.join(tomato_path, folder))
])

print("Classes:")
print(class_names)
print("Number of classes:", len(class_names))

... Classes:
['Tomato__Bacterial_spot',
 'Tomato__Early_blight',
 'Tomato__Late_blight',
 'Tomato__Leaf_Mold',
 'Tomato__Septoria_leaf_spot',
 'Tomato__Spider_mites_Two_spotted_spider_mite',
 'Tomato__Target_Spot',
 'Tomato__Tomato_mosaic_virus',
 'Tomato__Tomato_Yellow_Leaf_Curl_Virus',
 'Tomato__healthy']

Number of classes: 10
```

```

# 2. Create training and validation datasets

import tensorflow as tf
IMG_SIZE = (128, 128)
BATCH_SIZE = 32

train_ds = tf.keras.utils.image_dataset_from_directory(
    tomato_path,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    validation_split=0.2,
    subset="training",
    seed=42
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    tomato_path,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    validation_split=0.2,
    subset="validation",
    seed=42
)

Found 18160 files belonging to 10 classes.
Using 14528 files for training.
Found 18160 files belonging to 10 classes.
Using 3632 files for validation.

Classes: ['Tomato__Bacterial_spot',
'Tomato__Early_blight', 'Tomato__Late_blight',
'Tomato__Leaf_Mold',
'Tomato__Septoria_leaf_spot',
'Tomato__Spider_mites_Two_spotted_spider_mite',
'Tomato__Target_Spot',
'Tomato__Tomato_mosaic_virus',
'Tomato__Tomato_Yellow_Leaf_Curl_Virus', 'Tomato__healthy']

```

```
# 3. Use a smaller dataset for faster training
train_ds = train_ds.take(50)
val_ds = val_ds.take(15)
print("Small dataset created.")
```

Small dataset created.

```
1. # 4. Display sample images
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 8))
for images, labels in train_ds.take(1):
    for i in range(9):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        label = int(labels[i].numpy())
        plt.title(class_names[label])
        plt.axis("off")
plt.tight_layout()
plt.show()
```

Tomato___Bacterial_spot



Tomato___Early_blight



Tomato___Late_blight



Tomato___Leaf_Mold



Tomato___Septoria_leaf_spot



Tomato___Target_Spot



Tomato___Tomato_mosaic_virus



Tomato___healthy



```

1. # 5. Build CNN model
from tensorflow.keras import layers, models
num_classes = len(class_names)

model = models.Sequential([
    layers.Input(shape=(128, 128, 3)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])
model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 3,305,930 (12.61 MB)

Trainable params: 3,305,930 (12.61 MB)

Non-trainable params: 0 (0.00 B)



```
# 9. Evaluate the model
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import numpy as np
y_true, y_pred = [], []
for images, labels in val_ds:
    preds = model.predict(images, verbose=0)
    preds = np.argmax(preds, axis=1)
    y_true.extend(labels.numpy())
    y_pred.extend(preds)

accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='weighted')
recall = recall_score(y_true, y_pred, average='weighted')
f1 = f1_score(y_true, y_pred, average='weighted')
print("Accuracy :"; accuracy)
print("Precision: "; precision)
print("Recall : "; recall)
print("F1 Score : "; f1)
```

... Accuracy : 0.875
Precision : 0.8762
Recall : 0.875
F1 Score : 0.8748

```
# 10. Classification report
from sklearn.metrics import classification_report, confusion_matrix
print(classification_report(y_true, y_pred, target_names=class_names))
```

	precision	recall	f1-score	support
Tomato_Bacterial_spot	0.82	0.60	0.61	48
Tomato_Early_Blight	0.85	0.88	0.87	44
Tomato_Late_blight	0.86	0.84	0.85	45
Tomato_Leaf_Mold	0.88	0.86	0.87	45
Tomato__Septoria_leaf_spot	0.86	0.84	0.85	44
Tomato_Spider_web_Leaf_Spotted_Leaf_Rot	0.83	0.82	0.82	44
Tomato_Tomato_Leaf_Spot	0.88	0.89	0.89	45
Tomato_Tomato_Mosaic_Virus	0.91	0.89	0.90	45
Tomato_Tomato_Yellow_Leaf_Curl_Virus	0.89	0.87	0.88	45
Tomato_healthy	0.86	0.90	0.88	44
accuracy			0.88	444
macro avg	0.86	0.86	0.86	444
weighted avg	0.88	0.88	0.87	444